PHASE 9 OF 18

Exception Handling

The Throwable hierarchy, checked vs unchecked, try/catch/finally rules, multi-catch, try-with-resources, throw vs throws, propagation, custom and chained exceptions, common exceptions and errors, and best practices

IN THIS PHASE

9.1 What Is an Exception?
9.2 The Throwable Hierarchy
9.3 Checked vs Unchecked Exceptions
9.4 try and catch
9.5 The finally Block
9.6 try-with-resources (Java 7)
9.7 throw: Raising an Exception Yourself
9.8 throws: Declaring Exceptions
9.9 Exception Propagation
9.10 Useful Throwable Methods
9.11 Custom (User-Defined) Exceptions
9.12 Exception Chaining (Wrapping)
9.13 Exceptions and Overriding (Recap)
9.14 Common Exceptions: Cause and Fix
9.15 Important Errors
9.16 Best Practices
9.17 Predict the Output
9.18 Interview Questions
9.19 Quick Revision

9.1 What Is an Exception?

An **exception** is an unexpected event during program **execution** that disrupts the normal flow of instructions: dividing by zero, a missing file, a `null` reference, a lost DB connection. **Exception handling** lets the program respond and keep running (**graceful termination**) instead of crashing (**abnormal termination**).

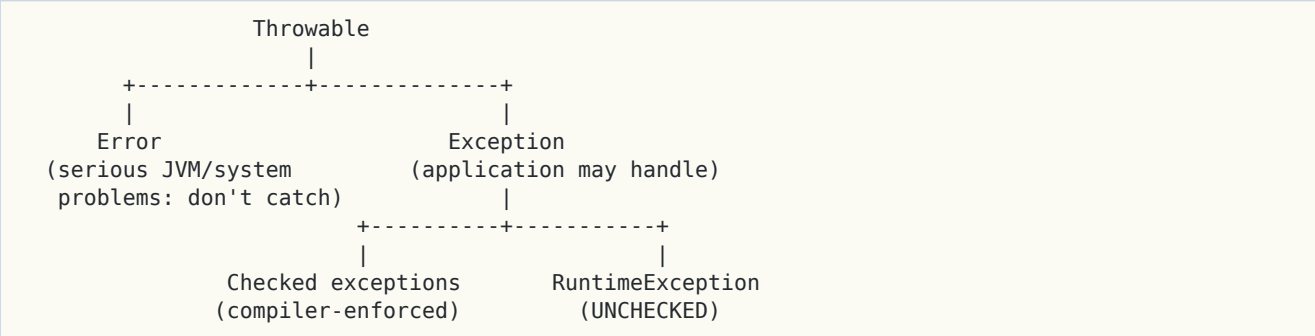
```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Start");
        int a = 10 / 0;                    // ArithmeticException thrown here
        System.out.println("End");         // never reached
    }
}
```

```
Start
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Demo.main(Demo.java:4)
```

Without a handler, the **default exception handler** prints the exception's name, message and **stack trace**, and terminates the thread.

Benefit of handling	Example
Prevents abnormal termination	Show "Service unavailable, try again" instead of crashing
Separates error handling from business logic	The happy path stays readable
Propagates errors to the right layer	A DAO throws, the service translates, the controller returns HTTP 4xx/5xx
Releases resources reliably	Files, DB connections and sockets are closed in `finally` or try-with-resources

9.2 The Throwable Hierarchy



Branch	Important subclasses
`Error`	`VirtualMachineError` → `OutOfMemoryError`, `StackOverflowError`; `LinkageError` → `NoClassDefFoundError`, `ExceptionInInitializerError`; `AssertionError`
Checked `Exception`s	`IOException` → `FileNotFoundException`, `EOFException`; `SQLException`; `ClassNotFoundException`; `InterruptedException`; `ParseException`; `CloneNotSupportedException`
`RuntimeException` (unchecked)	`ArithmeticException`; `NullPointerException`; `ClassCastException`; `IndexOutOfBoundsException` → `ArrayIndexOutOfBounds…`, `StringIndexOutOfBounds…`; `IllegalArgumentException` → `NumberFormatException`; `IllegalStateException`; `UnsupportedOperationException`; `ConcurrentModificationException`; `NoSuchElementException` → `InputMismatchException`; `ArrayStoreException`; `NegativeArraySizeException`; `DateTimeException`

CORRECTED FROM ORIGINAL NOTES

The hierarchy diagram in the original notes shows `OutOfMemoryError` and `StackOverflowError` as siblings of `VirtualMachineError`; they are its **subclasses**. Likewise, `FileNotFoundException` is a subclass of `IOException`, not a sibling.

Exception vs Error

	Exception	Error
Represents	Application-level problems	Serious JVM or system failures
Recoverable?	Usually	Usually not
Should you catch it?	Yes, where you can act on it	Generally no (log it at the top level and let the process fail or restart)
Examples	`IOException`, `SQLException`, `NullPointerException`	`OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`
Checked?	Checked or unchecked	Always unchecked

CORRECTED FROM ORIGINAL NOTES

The original notes say errors "cannot be handled", yet a later example catches `StackOverflowError`. Errors **can** technically be caught (they are `Throwable`s), but you usually **shouldn't**, because the JVM may be in an unreliable state.

9.3 Checked vs Unchecked Exceptions

	Checked exceptions	Unchecked exceptions
Which classes	`Exception` and its subclasses, **except** `RuntimeException`	`RuntimeException`, `Error` and their subclasses
Checked by	The **compiler**: you must handle or declare them	Nobody at compile time
Typically caused by	External conditions outside the program's control (files, network, DB)	**Programming bugs** (null, bad index, bad cast, bad argument)
Handling	**Handle or declare** rule: `try-catch` or `throws`	Optional. Fix the bug or validate input.
Examples	`IOException`, `FileNotFoundException`, `SQLException`, `ClassNotFoundException`, `InterruptedException`	`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`, `IllegalArgumentException`

```
FileReader fr = new FileReader("abc.txt");   // COMPILE ERROR: unreported exception
                                             // FileNotFoundException (catch or declare)
String s = null;
s.length();                                  // compiles fine, fails at RUNTIME with NPE
```

REMEMBER

"Checked" refers to **compile time**. Every exception, checked or not, actually occurs at **runtime**. `Exception` and `Throwable` are sometimes called *partially checked* because some of their subclasses are unchecked.

9.4 try and catch

```
try {
    // risky code
} catch (ExceptionType e) {
    // handling code: runs only if a matching exception is thrown
}
```

```
System.out.println("1. start");
try {
    int[] arr = new int[3];
    System.out.println("2. before");
    arr[5] = 10;                                  // exception: jump to catch
    System.out.println("3. after");               // SKIPPED
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("4. caught: " + e.getMessage());
}
System.out.println("5. end");                     // runs: graceful continuation
// 1. start  2. before  4. caught: Index 5 out of bounds for length 3  5. end
```

NOTE

Once an exception is thrown inside `try`, the **remaining statements in the try block are skipped**. Control never returns to the point of failure. Keep try blocks small and around only the risky lines.

Multiple catch blocks: child before parent

```
try {
    String s = args.length > 0 ? args[0] : null;
    int n = Integer.parseInt(s.trim());
    System.out.println(100 / n);
} catch (NullPointerException e) {
    System.out.println("No input");
} catch (NumberFormatException e) {
    System.out.println("Not a number");
} catch (ArithmeticException e) {
    System.out.println("Zero not allowed");
} catch (Exception e) {                    // most general LAST (catch-all)
    System.out.println("Unexpected: " + e);
}
```

The JVM checks catch blocks **top to bottom** and runs only the **first** matching one.

```
try { } catch (Exception e) { } catch (ArithmeticException e) { }
// COMPILE ERROR: exception ArithmeticException has already been caught (unreachable catch)
```

Multi-catch (Java 7)

```
try {
    // ...
} catch (NumberFormatException | ArithmeticException e) {   // same handling for both
    System.out.println("Bad input: " + e.getMessage());
    // e = new ArithmeticException();   // COMPILE ERROR: e is implicitly final
}
// catch (IOException | FileNotFoundException e)  -> COMPILE ERROR: types related by subclassing
```

PITFALL

Catching a checked exception that the try block can't throw is a compile error: `try { int x = 1; } catch (IOException e) { }` fails with "exception IOException is never thrown". `Exception`, `Throwable` and unchecked exceptions are exempt from this rule.

Nested try

```
try {
    try {
        int x = 10 / 0;
    } catch (ArithmeticException e) {
        System.out.println("Inner caught");
    }
    int[] a = new int[2];
    a[3] = 1;
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Outer caught");
}
// Inner caught
// Outer caught
```

If an inner `try` has no matching catch, the exception moves to the outer `try`'s catch blocks.

9.5 The finally Block

`finally` holds **cleanup code** that runs **whether or not an exception occurs**, and even when `try` or `catch` executes `return`, `break` or `continue`.

Scenario	try	catch	finally	Code after
No exception	Completes	Skipped	**Runs**	Runs

Scenario	try	catch	finally	Code after
Exception, matching catch	Stops at the exception	Runs	**Runs**	Runs
Exception, no matching catch	Stops at the exception	n/a	**Runs**	Skipped: the exception propagates
`return` inside try	Returns...	n/a	**Runs before the method actually returns**	n/a

```
FileReader fr = null;
try {
    fr = new FileReader("abc.txt");
    System.out.println(fr.read());
} catch (IOException e) {
    System.out.println("Read failed: " + e.getMessage());
} finally {
    if (fr != null) {
        try { fr.close(); } catch (IOException e) { /* log */ }   // verbose!
    }
}
```

`try-finally` without `catch` is legal: cleanup runs, and the exception still propagates. `try` alone (with neither catch nor finally) is not legal, except in the try-with-resources form.

When finally does NOT run

1. `System.exit()` (or `Runtime.halt()`) is called in `try` or `catch`.
2. The JVM crashes, or the process is killed (`kill -9`, power loss).
3. The `try`/`catch` never finishes: an infinite loop or a deadlock.
4. The thread is a daemon thread and the JVM shuts down.

return in finally: a trap

```
static int test() {
    try {
        return 1;
    } finally {
        return 2;          // OVERRIDES the try's return value (and swallows exceptions!)
    }
}
System.out.println(test());   // 2

static int counter() {
    int x = 10;
    try {
        return x;          // value 10 is saved here
    } finally {
        x = 20;            // changing the local doesn't affect the saved return value
    }
}
System.out.println(counter()); // 10
```

PITFALL

Never `return` (or `throw`) from `finally`. It silently discards any exception thrown in `try` and replaces the return value. Most IDEs and Sonar flag it.

final vs finally vs finalize

final	finally	finalize()
Keyword (modifier)	Block used with `try`	Method of `Object`
Constant variable, no override, no subclass	Cleanup that always runs	Called by GC before reclaiming (unreliable)
Widely used	Widely used (or try-with-resources)	**Deprecated** (Java 9, for removal in 18)

9.6 try-with-resources (Java 7)

Resources declared in the `try (...)` header are **closed automatically**, in **reverse order** of declaration, whether the block completes normally or not. A resource must implement `AutoCloseable` (or its sub-interface `java.io.Closeable`).

```
try (BufferedReader br = new BufferedReader(new FileReader("abc.txt"));
     BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        bw.write(line.toUpperCase());
        bw.newLine();
    }
} catch (IOException e) {
    System.out.println("I/O problem: " + e.getMessage());
}   // bw closed first, then br: no finally needed

// JDBC: Connection, PreparedStatement and ResultSet are all AutoCloseable
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement("SELECT balance FROM account WHERE id=?")) {
    ps.setLong(1, 1001L);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) System.out.println(rs.getBigDecimal(1));
    }
}
```

```
BufferedReader reader = new BufferedReader(new FileReader("abc.txt"));
try (reader) {                      // Java 9: an existing final/effectively-final variable
    System.out.println(reader.readLine());
}
```

Suppressed exceptions (verified output)

If the body throws **and** `close()` also throws, the body's exception is the one propagated. The close exception is attached to it as **suppressed** instead of being lost, as it would be with a hand-written `finally`.

```
class Res implements AutoCloseable {
    private final String name;
    Res(String name) { this.name = name; System.out.println("open " + name); }
    @Override public void close() {
        System.out.println("close " + name);
        throw new IllegalStateException("close failed: " + name);
    }
}

try (Res a = new Res("A"); Res b = new Res("B")) {
    throw new RuntimeException("body failed");
} catch (RuntimeException e) {
    System.out.println("caught: " + e.getMessage());
    for (Throwable s : e.getSuppressed()) System.out.println("  suppressed: " + s.getMessage());
}
```

```
open A
open B
close B                          <- reverse order
close A
caught: body failed
  suppressed: close failed: B
  suppressed: close failed: A
```

> **TIP**
> Use try-with-resources for **anything** that must be closed: streams, readers, writers, `Scanner`, JDBC objects, `ExecutorService` (Java 19+), HTTP responses. It's shorter, correct by default, and keeps suppressed exceptions.

9.7 throw: Raising an Exception Yourself

`throw` creates an exception **explicitly**, usually for validation or business rules. It throws **one** object of a type that extends `Throwable`.

```java
static String getName(int id) {
    if (id <= 0) {
        throw new IllegalArgumentException("id must be positive: " + id);
    }
    if (id != 100) {
        throw new RuntimeException("Invalid Id");
        // System.out.println("x");   // COMPILE ERROR: unreachable statement after throw
    }
    return "Rohit";
}
```

```java
catch (SQLException e) {
    log.error("DB failure", e);
    throw e;                        // RETHROW to the caller after partial handling
}

throw null;                         // compiles, but throws NullPointerException at runtime
```

9.8 throws: Declaring Exceptions

`throws` in a method signature **declares** that the method may throw these (usually checked) exceptions and **delegates** handling to the caller. It satisfies the compiler's *handle or declare* rule.

```java
void loadConfig(String path) throws IOException {        // declare: caller must deal with it
    try (BufferedReader br = Files.newBufferedReader(Path.of(path))) {
        System.out.println(br.readLine());
    }
}

void start() {
    try {
        loadConfig("app.properties");
    } catch (IOException e) {                            // handled here
        System.out.println("Using defaults: " + e.getMessage());
    }
}

public static void main(String[] args) throws Exception { }   // declare everything (demos only)
```

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say `throws` lets us "ignore" checked exceptions. Nothing is ignored. `throws` **passes responsibility to the caller**. If nobody up the call stack catches the exception, it still terminates the thread at runtime.

throw vs throws

throw	throws
Actually raises an exception	Declares possible exceptions
Used **inside** a method body	Used in the method **signature**
Followed by **one** exception object: `throw new X(...)`	Followed by one or more **class names**: `throws A, B`
Mainly for unchecked/business exceptions (any `Throwable` works)	Mainly for checked exceptions (unchecked allowed, for documentation)
Execution stops at the `throw`	Has no runtime effect by itself

9.9 Exception Propagation

When a method doesn't handle an exception, the JVM **unwinds the call stack**: it pops the current frame and looks for a handler in the **caller**, then the caller's caller, and so on. If it reaches the top of the thread unhandled, the default handler prints the stack trace and the thread dies.

```
class InvalidNumberException extends Exception {         // custom CHECKED exception
    InvalidNumberException(String msg) { super(msg); }
}

public class Demo {
    void m2(int a, int b) throws InvalidNumberException {
        System.out.println("m2() started");
        try {
            int c = a / b;
            if (c < 0) throw new InvalidNumberException("Negative value not allowed");
            System.out.println("Result = " + c);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception Occurred");
            throw e;                                     // rethrow
        } finally {
            System.out.println("m2() finally block executed");
        }
        System.out.println("m2() ended");
    }

    void m1(int a, int b) throws InvalidNumberException {
        System.out.println("m1() started");
        m2(a, b);
        System.out.println("m1() ended");
    }

    public static void main(String[] args) {
        System.out.println("main() started");
        try {
            new Demo().m1(10, 0);
        } catch (InvalidNumberException e) {
            System.out.println("User Defined Exception : " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Exception Handled In Main: " + e);
        }
        System.out.println("main() ended");
    }
}
```

```
main() started                    output with m1(10, 0)
m1() started
m2() started
Arithmetic Exception Occurred
m2() finally block executed         <- finally runs even while rethrowing
Exception Handled In Main: java.lang.ArithmeticException: / by zero
main() ended                         <- "m2() ended" and "m1() ended" never print

With m1(-10, 2): ... m2() finally block executed
                 User Defined Exception : Negative value not allowed
```

```
call stack                        propagation (unwinding)
+-----------+                     m2()  --X-->  m1()  --X-->  main()  -> caught
|   m2()    |  <- 10 / 0 here
|   m1()    |                     java.lang.ArithmeticException: / by zero
|   main()  |                         at Demo.m2(Demo.java:9)      <- origin (top)
+-----------+                         at Demo.m1(Demo.java:23)
                                      at Demo.main(Demo.java:31)   <- entry point
```

TIP

Read a stack trace **from the top**: the first line is the exception type and message, and the first `at` line is where it was thrown. Look for the first line in **your own package**. For chained exceptions, scroll to the last **Caused by:**, which is usually the root cause.

9.10 Useful Throwable Methods

Method	Returns
`getMessage()`	The detail message (may be `null`)
`toString()`	`ClassName: message`, as printed by `System.out.println(e)`
`printStackTrace()`	Prints the type, message and full call stack to `System.err`
`getCause()`	The wrapped underlying exception (chaining)
`getStackTrace()`	`StackTraceElement[]` for programmatic inspection
`getSuppressed()` / `addSuppressed()`	Exceptions suppressed during try-with-resources
`fillInStackTrace()`	Re-records the stack (rarely needed)

TIP

In real applications, don't call `e.printStackTrace()` or `System.out.println(e)`. Use a logger, which captures the message, the stack trace and the context: `log.error("Payment failed for txn {}", txnId, e);` (SLF4J: pass `e` as the **last** argument).

9.11 Custom (User-Defined) Exceptions

Create your own exception to represent a **domain-specific** problem with a meaningful name and extra data.

Extend	Type	Caller must handle?	Typical use
`Exception`	Checked	Yes (handle or declare)	Recoverable conditions the caller should consciously handle
`RuntimeException`	Unchecked	No	Business-rule violations and invalid state. **The most common choice in Spring Boot apps.**

```
public class InsufficientFundsException extends RuntimeException {
    private final String accountNo;
    private final BigDecimal shortfall;

    public InsufficientFundsException(String accountNo, BigDecimal shortfall) {
        super("Insufficient funds in " + accountNo + ", short by " + shortfall);
        this.accountNo = accountNo;
        this.shortfall = shortfall;
    }

    public InsufficientFundsException(String message, Throwable cause) {  // chaining ctor
        super(message, cause);
        this.accountNo = null;
        this.shortfall = null;
    }

    public String getAccountNo()     { return accountNo; }
    public BigDecimal getShortfall() { return shortfall; }
}

void withdraw(BigDecimal amount) {
    if (balance.compareTo(amount) < 0) {
        throw new InsufficientFundsException(accountNo, amount.subtract(balance));
    }
    balance = balance.subtract(amount);
}
```

NOTE

In Spring Boot, custom runtime exceptions are typically mapped to HTTP responses in one place with `@RestControllerAdvice` + `@ExceptionHandler` (for example `InsufficientFundsException` → 422, `NoDataFoundException` → 404). Validate those mappings in your API tests.

9.12 Exception Chaining (Wrapping)

When translating a low-level exception into a higher-level one, **pass the original as the cause** so the root cause isn't lost.

```
public Account findAccount(long id) {
    try {
        return jdbcFind(id);
    } catch (SQLException e) {
        throw new DataAccessFailure("Could not load account " + id, e);   // keep the cause!
    }
}
```

```
Exception in thread "main" DataAccessFailure: Could not load account 1001
    at AccountRepo.findAccount(AccountRepo.java:12)
    ...
Caused by: java.sql.SQLException: Connection refused
    at ...
```

PITFALL

`throw new DataAccessFailure("failed");` without the cause, or `throw new X(e.getMessage())`, throws away the original stack trace. Debugging production issues becomes guesswork.

9.13 Exceptions and Overriding (Recap)

Parent method declares	Overriding method may declare
Nothing	No **checked** exceptions (any unchecked are fine)
`throws IOException`	Nothing, `IOException`, or a subclass such as `FileNotFoundException`
`throws IOException`	**Not** `Exception` or an unrelated checked exception such as `SQLException`
Anything	Any unchecked exceptions

Constructors are not overridden, but a subclass constructor must declare (at least) the checked exceptions of the `super(...)` constructor it calls, because it can't catch them around `super(...)`.

9.14 Common Exceptions: Cause and Fix

Exception	Type	Typical cause	Typical fix
`NullPointerException`	Unchecked	Calling a member on `null`	Validate inputs, `Objects.requireNonNull`, `Optional`, null-safe `equals`
`ArithmeticException`	Unchecked	Integer `/ 0`; non-terminating `BigDecimal.divide`	Check the divisor; pass scale and `RoundingMode`
`ArrayIndexOutOfBoundsException`	Unchecked	Index outside `0..length-1`	Loop with `i < arr.length`; check `args.length`
`StringIndexOutOfBoundsException`	Unchecked	Bad `charAt`/`substring` index	Check `length()` first
`NumberFormatException`	Unchecked	`Integer.parseInt("abc")`, spaces, empty string	`trim()`, validate with regex, catch it
`ClassCastException`	Unchecked	Invalid downcast	Check with `instanceof` first
`IllegalArgumentException`	Unchecked	Invalid argument (e.g. `Thread.sleep(-100)`)	Validate early; throw it yourself for bad args
`IllegalStateException`	Unchecked	Wrong object state (using a closed `Scanner`, `Iterator.remove()` twice)	Check the state machine
`UnsupportedOperationException`	Unchecked	Modifying `List.of(...)` or an `Arrays.asList` size	Copy into an `ArrayList`
`ConcurrentModificationException`	Unchecked	Modifying a collection while iterating over it	`Iterator.remove()`, `removeIf`, concurrent collections
`NoSuchElementException`	Unchecked	`iterator.next()` with none left, `Optional.get()` on empty	`hasNext()`, `orElse`/`orElseThrow`
`InputMismatchException`	Unchecked	`Scanner.nextInt()` reads "abc"	`hasNextInt()` first
`ArrayStoreException`	Unchecked	Wrong type stored in a covariant array	Use generics/collections
`NegativeArraySizeException`	Unchecked	`new int[-5]`	Validate the size
`FileNotFoundException`	Checked	Wrong path, or no permission	Check `Files.exists`; use absolute/config paths
`IOException`	Checked	Any I/O failure	try-with-resources, retry, report
`SQLException`	Checked	DB errors: bad SQL, constraints, connection	Wrap and translate; check the SQLState
`ClassNotFoundException`	Checked	`Class.forName("x")` can't find the class	Fix the classpath or dependency
`InterruptedException`	Checked	Thread interrupted during `sleep`/`wait`/`join`	Restore the flag: `Thread.currentThread().interrupt()`
`ParseException`	Checked	`SimpleDateFormat.parse` with bad input	Prefer `java.time` (`DateTimeParseException`, unchecked)
`CloneNotSupportedException`	Checked	`clone()` on a non-`Cloneable` class	Implement `Cloneable` or use a copy constructor

Helpful NullPointerException messages (Java 14+)

```
Customer c = new Customer();
c.getAddress().getCity().length();
// Java 8:   java.lang.NullPointerException
// Java 14+: Cannot invoke "String.length()" because the return value of
//           "Address.getCity()" is null
```

```
this.name = Objects.requireNonNull(name, "name must not be null");   // fail fast, clear message
String city = Optional.ofNullable(c.getAddress()).map(Address::getCity).orElse("Unknown");
```

9.15 Important Errors

Error	Cause	What to do
`StackOverflowError`	Infinite or very deep recursion (each call adds a stack frame)	Fix the base case, convert to iteration, or increase `-Xss`
`OutOfMemoryError: Java heap space`	Too many live objects, or a memory leak (static collections, caches, unclosed resources)	Heap dump (`-XX:+HeapDumpOnOutOfMemoryError`), analyse with Eclipse MAT, fix the leak, tune `-Xmx`
`OutOfMemoryError: Metaspace`	Too many classes loaded (class-loader leaks, heavy proxies)	Find the leak; tune `-XX:MaxMetaspaceSize`
`OutOfMemoryError: unable to create native thread`	Too many threads	Use thread pools or virtual threads
`NoClassDefFoundError`	The class was present at **compile** time but is missing or failed to initialise at **runtime**	Fix the runtime classpath, JAR conflicts, or the static-init failure
`ExceptionInInitializerError`	An exception was thrown in a static initializer or static field initialisation	Fix the static block (check its cause)
`AssertionError`	A failed `assert` (enabled with `-ea`), or thrown explicitly	Fix the violated invariant

INTERVIEW

`ClassNotFoundException` vs `NoClassDefFoundError`: the former is a **checked exception** thrown when code **dynamically** loads a class by name (`Class.forName`, `loadClass`) and it's not found. The latter is an **Error** thrown when a class that was available at **compile time** can't be found or initialised when the JVM needs it at **runtime**.

```
void m1() { m1(); }        // infinite recursion
try {
    m1();
} catch (StackOverflowError e) {
    System.out.println("StackOverflowError Handled");   // possible, but fix the recursion
}
```

9.16 Best Practices

1. **Catch specific exceptions**, not `Exception`/`Throwable`, except at a top-level boundary (controller advice, the main loop, a job runner).
2. **Never swallow exceptions**: an empty `catch {}` hides failures. At minimum, log with the stack trace.
3. **Throw early, catch late**: validate inputs at method entry; handle where you can actually do something meaningful.
4. Use **try-with-resources** for every closable resource.
5. **Preserve the cause** when wrapping (`new X(msg, e)`).
6. Use **unchecked exceptions for programming errors and business-rule violations**; checked exceptions for recoverable external conditions the caller must address.
7. Don't use exceptions for **normal control flow** (such as ending a loop), because they are slow and hard to read.
8. Write **meaningful messages** with context (IDs, values), but **never include secrets** (passwords, card numbers, tokens).
9. Don't `return` or `throw` from `finally`.
10. On `InterruptedException`, either propagate it or restore the interrupt flag with `Thread.currentThread().interrupt()`.
11. Prefer standard exceptions where they fit: `IllegalArgumentException`, `IllegalStateException`, `UnsupportedOperationException`, `NoSuchElementException`.
12. In tests, assert exceptions precisely: JUnit 5 `assertThrows(InsufficientFundsException.class, () -> acc.withdraw(big))`, plus checks on the message or fields.

9.17 Predict the Output

```
static int f() {
    try { throw new RuntimeException(); }
    catch (RuntimeException e) { return 1; }
    finally { System.out.print("F "); }
}
System.out.println(f());                          // F 1

try {
    System.out.print("A ");
    int x = 5 / 0;
    System.out.print("B ");
} catch (ArithmeticException e) {
    System.out.print("C ");
} finally {
    System.out.print("D ");
}
System.out.println("E");                          // A C D E

try {
    throw new IllegalArgumentException("bad");
} catch (RuntimeException e) {
    System.out.println(e);                        // java.lang.IllegalArgumentException: bad
}

static int g() { int x = 1; try { return x; } finally { x = 99; } }
System.out.println(g());                          // 1

try { Object o = "s"; Integer i = (Integer) o; }
catch (ClassCastException e) { System.out.println("CCE"); }      // CCE

String s = null;
try { s.length(); } catch (Exception e) { System.out.println(e.getClass().getSimpleName()); }
                                                  // NullPointerException
```

9.18 Interview Questions

Q. What is the difference between checked and unchecked exceptions?

Checked exceptions (subclasses of `Exception` excluding `RuntimeException`) are verified by the compiler and must be caught or declared. Unchecked exceptions (`RuntimeException` and `Error` subclasses) aren't enforced by the compiler and usually indicate programming bugs.

Q. What is the difference between an Exception and an Error?

Exceptions represent application-level conditions that can often be handled. Errors represent serious JVM or system problems (`OutOfMemoryError`, `StackOverflowError`) that applications generally shouldn't try to handle.

Q. Can we have try without catch?

Yes, `try-finally`, or try-with-resources, which needs neither catch nor finally. A bare `try { }` alone isn't allowed.

Q. Does finally always execute?

Almost always: after normal completion, after a caught or uncaught exception, and even after `return`. It doesn't run if `System.exit()` is called, the JVM crashes or is killed, or the try never completes (infinite loop or deadlock).

Q. What happens if both try and finally have return statements?

The `finally` return wins. It overrides the try's value and also discards any pending exception, which is why it's a bad practice.

Q. Why must catch blocks go from specific to general?

Catch blocks are matched top-down. A general type placed first would catch everything, so later specific blocks become unreachable, which is a compile error.

Q. What is try-with-resources? What are suppressed exceptions?

A Java 7 construct that automatically closes `AutoCloseable` resources in reverse order. If the body and `close()` both throw, the body's exception propagates and the close exceptions are attached to it via `getSuppressed()`.

Q. What is the difference between `throw` and `throws`?

`throw` raises a single exception object inside a method. `throws` declares in the signature which exceptions a method may pass to its caller.

Q. What is exception propagation?

When a method doesn't handle an exception, it moves up the call stack to the caller, and so on, until a matching catch is found or the thread terminates with a stack trace.

Q. How do you create a custom exception? Checked or unchecked?

Extend `Exception` for a checked exception or `RuntimeException` for an unchecked one. Provide constructors with a message and a cause, and add domain fields if useful. Business-rule exceptions in modern apps are usually unchecked.

Q. What is exception chaining?

Wrapping a lower-level exception as the cause of a higher-level one (`new ServiceException(msg, e)`), so that the full root cause and stack trace are preserved ("Caused by: ...").

Q. What is the difference between ClassNotFoundException and NoClassDefFoundError?

The first is a checked exception from dynamic loading by name when the class isn't found. The second is an Error raised when a class that existed at compile time is missing or failed to initialise at runtime.

Q. What are the exception rules for overriding methods?

An overriding method may throw the same, narrower or no checked exceptions, never broader or new ones. Unchecked exceptions are unrestricted.

9.19 Quick Revision

- `Throwable` → `Error` (don't catch) and `Exception` → checked types plus `RuntimeException` (unchecked).
- **Checked** = compiler-enforced: *handle or declare*. **Unchecked** = `RuntimeException` and `Error`, usually bugs.
- Lines after the exception point inside `try` are skipped; only the first matching catch runs.
- Order catch blocks child → parent; multi-catch `A | B` (unrelated types, `e` final).
- `finally` runs except on `System.exit`, a crash or a hang. Never `return` from `finally`.
- try-with-resources: `AutoCloseable`, closed in reverse order, suppressed exceptions kept. Java 9 allows existing effectively-final variables.
- `throw` = raise one object; `throws` = declare and delegate. Neither ignores anything.
- Propagation unwinds the stack; read stack traces top-down and look for "Caused by".
- Custom exceptions: extend `RuntimeException` (business rules) or `Exception` (recoverable); keep the cause.
- Overriding: no new or broader checked exceptions.
- `ClassNotFoundException` (checked, dynamic loading) vs `NoClassDefFoundError` (Error, missing at runtime).
- Best practice: specific catches, no empty catches, log with context (no secrets), restore the interrupt flag.